

Tipus recursius de dades

Departament de Ciències de la Computació

Universitat Politècnica de Catalunya



UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Escola Politècnica Superior d'Enginyeria
de Vilanova i la Geltrú

Índex

1 Punters i memòria dinàmica

2 Introducció als tipus recursius de dades

3 Definició d'una estructura de dades recursiva

- Implementació de piles
- Implementació de cues

Índex

- 1 Punters i memòria dinàmica
- 2 Introducció als tipus recursius de dades
- 3 Definició d'una estructura de dades recursiva

Punters (I)

En C++, per cada tipus T hi ha un altre tipus “punter a T ”

Una variable de tipus “punter a T ” pot contenir:

- 1 una referència a una variable o objecte de tipus T ,
- 2 o un valor especial `NULL`,
- 3 o res vàlid, si no ha estat inicialitzada

Punters (II)

Operadors

- ➊ $T^* p$: Declaració de la variable p com a “punter a T ”
- ➋ $*p$: Objecte referenciat pel punter p
- ➌ $->$: Composició de $*$ i el selector $.$ de `struct/class`
- ➍ $\&v$: Referència a v , de tipus “punter a tipus de v ”
- ➎ `new`, `delete`: Creació i destrucció de memòria dinàmica

Punters (III)

Exemple

```
int x;    // declara i reserva memòria
          // per una variable x de tipus int
int* p;   // p és un punter (no inicialitzat)
          // a objectes de tipus int

p = &x;
x = 5;
cout << *p << endl;    // escriu 5
*p = 3;
cout << x << endl;     // escriu 3
```

Punters (IV)

Observacions

- Si p **no referencia cap objecte**, és a dir si,
 - $p == \text{NULL}$, o
 - p no inicialitzat→ **ERROR** a l'accedir a $*p$

Punters (V)

Observacions

```
Punt x;           // declara i reserva memòria
                  // per un objecte x de tipus Punt
Punt* ppunt;      // ppunt és un punter (no inicialitzat)
                  // a objectes de tipus Punt
```

- `x` sempre referenciarà el mateix objecte
- `*ppunt` pot anar referenciant diferents objectes quan canviem el valor de `ppunt`
- Quan fem `ppunt = &x`, tenim un objecte amb dos noms, `*ppunt` i `x`. Se'n diu **aliasing**, pot ser útil però també perillós.

Punters (VI)

Exemple

```
int x = 1;
int y = 2;
int* p = &x;
int* q = &y;
cout << x << "_" << y << endl; // escriu "1 2"
*q = *p;
cout << x << "_" << y << endl; // escriu "1 1"
*q = 3;
cout << x << "_" << y << endl; // escriu "1 3"
q = p;
*q = 4;
cout << x << "_" << y << endl; // escriu "4 3"
// en aquest punt, x té tres noms: x, *p i *q
```

Punters i structs

És freqüent tenir un **punter a un struct** o un **punter a un objecte d'una classe** i voler accedir a un camp de l'struct apuntat o accedir als atributs i mètodes de l'objecte apuntat.

Notació: `p->camp` equival a `(*p).camp`
`p->mètode(...)` equival a `(*p).mètode(...)`

Exemple:

```
Punt* ppunt;  
Punt p;  
cin >> p;  
ppunt = &p;  
if (ppunt->coordenadax() > 0.0)  
    cout << ppunt->coordenaday() << endl;
```

Recordeu que ja ho hem vist quan fem servir el `this` (punter al paràmetre implícit). Exemple: `this->x`

Definir un punter

Quan declarem un punter $T^* \ p$; p està indefinit!

Com el podem definir:

- 1 Fent que apunti a un objecte del tipus T ja existent:
 - assignant-li l'adreça de l'objecte: $p = \&x$;
 - assignant-li un punter a l'objecte: $p = q$;
- 2 O donant-li el valor `NULL` per indicar que “no referencia res”
- 3 O reservant memòria perquè apunti a un **objecte nou**:
 $p = \text{new } T$;
 - aquest objecte no tindrà nom propi, només $*p$
 - queda **inaccessible!** si modifiquem p i no hi ha cap altre punter que l'apunti

new i delete (I)

Operacions de gestió de memòria dinàmica:

- `new T`: Reserva memòria dinàmica per a un nou objecte, li aplica el constructor de `T` i retorna un punter a ell.
- `delete p`: Aplica el destructor del tipus a l'objecte apuntat per `p` i allibera la memòria que ocupa (“esborra” l'objecte).

new i delete (I)

Operacions de gestió de memòria dinàmica:

- `new T`: Reserva memòria dinàmica per a un nou objecte, li aplica el constructor de `T` i retorna un punter a ell.
- `delete p`: Aplica el destructor del tipus a l'objecte apuntat per `p` i allibera la memòria que ocupa (“esborra” l'objecte).
- Atenció: `delete p` **NO esborra el punter** `p` sinó que **esborra l'objecte apuntat** per `p`
- el valor de `p` després de `delete p` és **indefinit**.

new i delete (II)

Exemples

```
class T {  
    int camp1;  
    bool camp2;  
};  
void f (...) {  
    T x; // es crida al constructor de T  
    x.camp1 = 20; x.camp2 = true;  
    T* p = new T; // p apunta a un objecte nou;  
                // crida al constructor de T  
    p->camp1 = 30; p->camp2 = false;  
    ...  
    delete p; // es crida al destructor de T  
              // i s'allibera *p; p indefinit  
    // aquí es crida automàticament el destructor de T  
    // amb l'objecte local x (surta àmbit de visibilitat)  
}
```

new i delete (III)

Errors

- Deixar memòria sense alliberar (objectes dinàmics sense esborrar)
→ *memory leaks*
- Accedir a memòria ja alliberada (objectes esborrats). Vigileu amb l'*aliasing* → *dangling references*
- `delete` de memòria no creada amb `new`
- Confondre `p = NULL` amb `delete p`
 - els dos s'hauran d'usar, però en circumstàncies diferents.

new i delete (IV)

Exemples d'errors

```
void f (...) {
    T x; ... // assignar valor a x ...
    T* p = &x;
    T* q = new T;
    T* r = q; // r i q apunten al mateix valor
    T* s = new T;
    ...
    delete p; // ERROR: *p no creat amb new
    delete q; // OK
    if ((q->camp1 == 0) {...} // ERROR: *q no existeix
    if (q == NULL) {...} // PERILL: q indefinit
    r->camp1 = 3; // ERROR: r indefinit,
                  // *r ha estat alliberat amb delete q
    // ERROR: no fem delete s i *s es fa inaccessible: leak!
}
```


Assignació, còpia i destrucció (I)

Assignació i còpia:

- L'assignació entre punters a objectes no implica una còpia d'objectes.
- Cal definir un **constructor per còpia** per al tipus/classe. Usarà `new`. El constructor per còpia **per defecte** crea un nou objecte a partir d'un altre, copiant atribut a atribut.
- Sovint es redefineix també l'operador `=`, que per defecte fa assignació atribut a atribut de l'objecte origen a l'objecte destinació. Copiar/assignar atributs que siguin punters → *aliasing!*

Assignació, còpia i destrucció (II)

Destrucció/esborrament:

- `delete p`: no s'aplica recursivament a les components de l'objecte apuntat per `p` (*memory leaks*).
- Cal definir el **destructor** de la classe de manera que **s'alliberin tots els objectes creats amb memòria dinàmica** que s'hagin usat per representar un objecte de la classe. Usarà `delete`.

Pas de punters com a paràmetres

Pas d'un objecte x que conté punters, com a paràmetre d'entrada:

- Pas per valor:
 - Fa servir el constructor per còpia, hem de definir-lo si el constructor per còpia per defecte no serveix.
 - Si l'objecte x té atributs que són punters, el constructor per còpia per defecte ens porta a una situació d'*aliasing*.
- Pas per referència constant: x no es canviarà, però no es garanteix que no es modifiquin objectes apuntats per components de x .

Índex

- 1 Punters i memòria dinàmica
- 2 Introducció als tipus recursius de dades
- 3 Definició d'una estructura de dades recursiva

Tipus recursius de dades (I)

Operació recursiva

Operació que es crida a ella mateixa per aplicar el mateix tractament a tots o una part dels elements dels seus paràmetres.

Tipus recursius de dades (I)

Operació recursiva

Operació que es crida a ella mateixa per aplicar el mateix tractament a tots o una part dels elements dels seus paràmetres.

Perquè no hi hagi un nombre infinit de crides, cal que:

Tipus recursius de dades (I)

Operació recursiva

Operació que es crida a ella mateixa per aplicar el mateix tractament a tots o una part dels elements dels seus paràmetres.

Perquè no hi hagi un nombre infinit de crides, cal que:

- 1 Hi hagi com a mínim un cas directe (opció sense crida recursiva).

Tipus recursius de dades (I)

Operació recursiva

Operació que es crida a ella mateixa per aplicar el mateix tractament a tots o una part dels elements dels seus paràmetres.

Perquè no hi hagi un nombre infinit de crides, cal que:

- 1 Hi hagi com a mínim un cas directe (opció sense crida recursiva).
- 2 Els paràmetres de l'operació decreixin a cada crida recursiva.

Tipus recursius de dades (I)

Operació recursiva

Operació que es crida a ella mateixa per aplicar el mateix tractament a tots o una part dels elements dels seus paràmetres.

Perquè no hi hagi un nombre infinit de crides, cal que:

- 1 Hi hagi com a mínim un cas directe (opció sense crida recursiva).
- 2 Els paràmetres de l'operació decreixin a cada crida recursiva.

Però, què és un **tipus recursiu** de dades?

Tipus recursius de dades (II)

Quan hem vist les estructures de dades lineals i arborescents, ja les hem tractat de forma recursiva. Exemples:

Tipus recursius de dades (II)

Quan hem vist les estructures de dades lineals i arborescents, ja les hem tractat de forma recursiva. Exemples:

- Piles: una pila o bé és buida o bé és `push`(una altra pila, valor).
- Cues, llistes: ídem, amb les seves operacions.
- Arbres: un arbre o bé és buit, o bé és `arbre`(valor, arbre1, arbre2).

Tipus recursius de dades (II)

Quan hem vist les estructures de dades lineals i arborescents, ja les hem tractat de forma recursiva. Exemples:

- Piles: una pila o bé és buida o bé és `push`(una altra pila, valor).
- Cues, llistes: ídem, amb les seves operacions.
- Arbres: un arbre o bé és buit, o bé és `arbre`(valor, arbre1, arbre2).

Si poguéssim definir aquests tipus de dades de forma recursiva, podríem:

- Establir una correspondència entre la definició natural i la recursiva.
- No posar límits *a priori* en la mida màxima.

Tipus recursius de dades (III)

Implementació en C++, és possible?

```
class stack<T> {  
    private:  
        bool es_buida;  
        T valor;  
        stack<T> resta_pila;  
    public:  
        ...  
}
```

Tipus recursius de dades (III)

Implementació en C++, és possible?

```
class stack<T> {  
    private:  
        bool es_buida;  
        T valor;  
        stack<T> resta_pila;  
    public:  
        ...  
}
```

Problema: En C++, quan es crea un objecte es crida recursivament als constructors de totes les seves components. Procés infinit!

Tipus recursius de dades (IV)

Solució (exemple amb la Pila)

```
template <class T> class Pila {  
    private:  
        // tipus privat nou  
        struct node_pila {  
            T info;  
            node_pila* seguent; // ← recursivitat  
        };  
        int altura;           // guardada un sol cop  
        node_pila* primer_node; // primer d'una cadena  
                                // de nodes  
        ...                  // especificació d'operacions privades  
    public:  
        ...                  // especificació d'operacions públiques  
}
```

Tipus recursius de dades (IV)

Solució (exemple amb la Pila)

```
template <class T> class Pila {
private:
    // tipus privat nou
    struct node_pila {
        T info;
        node_pila* seguent; // <-- recursivitat
    };
    int altura;             // guardada un sol cop
    node_pila* primer_node; // primer d'una cadena
                           // de nodes
    ...                     // especificació d'operacions privades
public:
    ...                     // especificació d'operacions públiques
}
```

Els punters `seguent` no s'inicialitzen automàticament: no es creen objectes recursivament quan es crea una `Pila`.

Índex

- 1 Punters i memòria dinàmica
- 2 Introducció als tipus recursius de dades
- 3 Definició d'una estructura de dades recursiva**
 - Implementació de piles
 - Implementació de cues

Definició d'una estructura de dades recursiva

La definirem en dos nivells:

Definició d'una estructura de dades recursiva

La definirem en dos nivells:

- Nivell superior, classe (atributs) amb:
 - La informació global de l'estructura (la que no volem que es repeteixi per cada element).
 - Punters a alguns elements distingits (el primer, l'últim, etc, segons el que calgui).

Definició d'una estructura de dades recursiva

La definirem en dos nivells:

- Nivell superior, classe (atributs) amb:
 - La informació global de l'estructura (la que no volem que es repeteixi per cada element).
 - Punters a alguns elements distingits (el primer, l'últim, etc, segons el que calgui).
- Nivell inferior, `struct` privat que defineix nodes enllaçats per punters:
 - Informació d'un i només un element de l'estructura.
 - Punter a un o més nodes "següents".

Avantatges de les estructures de dades recursives

- Correspondència natural amb una definició recursiva abstracta.
- No cal fixar a priori un nombre màxim d'elements.
- Es pot anar demanant memòria per als nous nodes a mesura que s'hi volen afegir elements
- Eficiència: modificant enllaços entre nodes podem:
 - inserir o esborrar elements - sense moure els altres
 - moure parts senceres de l'estructura - sense fer còpies

Índex

1 Punters i memòria dinàmica

2 Introducció als tipus recursius de dades

3 Definició d'una estructura de dades recursiva

- Implementació de piles
- Implementació de cues

Implementació de piles (I)

```
template <class T> class Pila {  
private:  
    // tipus privat nou  
    struct node_pila {  
        T info;  
        node_pila* seguent;    // NULL indica final de cadena  
    };  
  
    int altura;                // guardada un sol cop  
    node_pila* primer_node;    // primer d'una cadena de nodes  
  
    ... // especificació d'operacions privades  
  
public:  
  
    ... // especificació d'operacions públiques  
};
```

Implementació de piles (II)

- La pila buida tindrà `altura 0` i `primer_node nul`.

Implementació de piles (II)

- La pila buida tindrà `altura 0` i `primer_node nul`.
- El cim d'una pila no buida serà el seu `primer_node`.

Implementació de piles (II)

- La pila buida tindrà `altura 0` i `primer_node nul`.
- El cim d'una pila no buida serà el seu `primer_node`.
- Cal definir operacions perquè `copiar` i `esborrar` siguin segures. Han de recórrer **tota** l'estructura.

Mètodes públics: constructors

```
Pila()  
/* Pre: cert */  
/* Post: El resultat és una pila sense cap element */  
{  
    altura = 0;  
    primer_node = NULL;  
}  
  
// constructor per còpia  
Pila(const Pila& original)  
/* Pre: cert */  
/* Post: El resultat és una còpia d'original */  
{  
    altura = original.altura;  
    primer_node = copia_node_pila(original.primer_node);  
}
```

Mètodes públics: destructor

```
~Pila()  
// Destructor: esborra automàticament els objectes  
// locals en sortir d'un àmbit de visibilitat  
{  
    esborra_node_pila(primer_node);  
}
```

Mètodes públics: consultors (I)

```
T cim() const
/* Pre: El p.i. és una pila no buida
   (<=> primer_node != NULL) */
/* Post: El resultat és el darrer element afegit
   al p.i. */
{
    return primer_node->info;
}
bool es_buida() const
/* Pre: cert */
/* Post: El resultat indica si el p.i. és una pila
   buida o no */
{
    return primer_node==NULL;
}
```

Mètodes públics: consultors (II)

```
int mida() const
/* Pre: cert */
/* Post: El resultat és el nombre d'elements
        del p.i. */
{
    return altura;
}
```

Mètodes públics: modificadors (I)

```
void empilar(const T& x)
/* Pre: cert */
/* Post: El p.i. és com el p.i. original amb x
         afegit com a darrer element */
{
    node_pila* aux;
    aux = new node_pila; // reserva espai nou element
    aux->info = x;
    aux->seguent = primer_node;
    primer_node = aux;
    ++altura;
}
```

Mètodes públics: modificadors (II)

```
void desempilar()  
/* Pre: El p.i. és una pila no buida  
   (<=> primer_node != NULL) */  
/* Post: El p.i. és com el p.i. original però sense  
   el darrer element afegit */  
{  
    node_pila* aux;  
    aux = primer_node; // conserva l'accés al primer  
                       // node abans d'avançar  
    primer_node = primer_node->seguent; // avança  
    delete aux; // allibera l'espai de l'antic cim  
    —altura;  
}
```


Mètodes públics: modificadors (III)

```
void p_buida()  
/* Pre: cert */  
/* Post: El p.i. passa a ser una pila sense elements  
i qualsevol contingut anterior del p.i. ha estat  
esborrat */  
{  
    esborra_node_pila(primer_node);  
    altura = 0;  
    primer_node= NULL;  
}
```

Mètodes públics: modificadors (IV)

Redefinició de l'operador assignació

```
stack<int> p1, p2, p3;
...
p1 = p2 = p3;
```

L'assignació en C++ és un operador: una funció que retorna un valor, amb paràmetre implícit que queda modificat, i un paràmetre explícit no modificable.

Return (*this): necessari per encadenaments d'assignacions.

```
Pila& operator=(const Pila& original)
/* Pre: cert */
/* Post: El p.i. passa a ser una còpia d'original i el contingut
   anterior del p.i. ha estat esborrat (excepte si el p.i. i
   original ja eren el mateix objecte) */
{
    if (this != &original) { // si no són el mateix objecte
        altura = original.altura;
        esborra_node_pila(primer_node); // si no, leak!
        primer_node = copia_node_pila(original.primer_node);
    }
    return *this;
}
```

Mètodes privats: per copiar cadenes de nodes

```
static node_pila* copia_node_pila(node_pila* m)
/* Pre: cert */
/* Post: Si m és NULL, el resultat és NULL; sinó,
    el resultat apunta al primer node d'una cadena
    de nodes que són còpia de la cadena que té el
    node apuntat per m com a primer */
{
    node_pila* n;
    if (m == NULL) n = NULL;
    else {
        n = new node_pila;
        n->info = m->info;
        n->seguent = copia_node_pila(m->seguent);
    }
    return n;
}
```

Exercici: Feu la versió iterativa d'aquesta funció.

Mètodes privats: per esborrar cadenes de nodes

```
static void esborra_node_pila (node_pila* m)
/* Pre: cert */
/* Post: No fa res si m és NULL, sinó allibera
   espai dels nodes de la cadena que té el node
   apuntat per m com a primer */
{
    if (m != NULL) {
        esborra_node_pila(m->seguent);
        delete m;
    }
}
```

Exercici: Feu la versió iterativa d'aquesta funció.

Comentaris

- El constructor còpia i l'assignació s'han de programar expressament,
 - volem còpies reals dels objectes, no assignacions de punters.

Comentaris

- El constructor còpia i l'assignació s'han de programar expressament,
 - volem còpies reals dels objectes, no assignacions de punters.
- També s'ha de programar el destructor,
 - per visitar tots els nodes i esborrar-los un a un.

Comentaris

- El constructor còpia i l'assignació s'han de programar expressament,
 - volem còpies reals dels objectes, no assignacions de punters.
- També s'ha de programar el destructor,
 - per visitar tots els nodes i esborrar-los un a un.
- A PRO1, només permetem que una funció retorni T& (p.e. Pila&) per operadors d'assignació sobrecarregats.
- Fora de la capçalera d'una funció, només permetem & per veure si dos objectes són iguals.

Índex

1 Punters i memòria dinàmica

2 Introducció als tipus recursius de dades

3 Definició d'una estructura de dades recursiva

- Implementació de piles

- Implementació de cues

Implementació de cues (I)

- Atribut per la longitud (o mida) de la cua.

Implementació de cues (I)

- Atribut per la longitud (o mida) de la cua.
- Ara necessitem accedir tant al primer element (per consultar-lo o esborrar-lo) com a l'últim (per afegir un nou element).

Implementació de cues (II)

```
template <class T> class Cua {  
private:  
    struct node_cua {  
        T info;  
        node_cua* seguent;  
    };  
    int longitud;  
    node_cua* primer_node;  
    node_cua* ultim_node;  
    ...  
    //especificació i implementació d'ops. privades  
  
public:  
    ...  
    //especificació i implementació d'ops. públiques  
};
```

Implementació de cues (III)

- La cua buida tindrà `longitud 0` i dos punters a nodes nuls.

Implementació de cues (III)

- La cua buida tindrà `longitud 0` i dos punters a nodes nuls.
- El primer (o *front()*) d'una cua no buida serà el seu `primer_node`.

Implementació de cues (III)

- La cua buida tindrà `longitud 0` i dos punters a nodes nuls.
- El primer (o *front()*) d'una cua no buida serà el seu `primer_node`.
- Caldrà definir operacions privades perquè **copiar** i **esborrar** siguin segures.

Mètodes públics: constructors

```
Cua()  
/* Pre: cert */  
/* Post: El resultat és una cua sense elements */  
{  
    longitud = 0;  
    primer_node = NULL;  
    ultim_node = NULL;  
}  
Cua(const Cua& original)  
/* Pre: cert */  
/* Post: El resultat és una còpia d'original */  
{  
    longitud = original.longitud;  
    primer_node = copia_node_cua(original.primer_node, ultim_node);  
}
```

Mètodes públics: destructor

```
~Cua()  
// Destructor: Esborra automàticament els objectes  
// locals en sortir d'un àmbit de visibilitat  
{  
    esborra_node_cua(primer_node);  
}
```


Mètodes públics: consultors (I)

```
T primer() const
/* Pre: El p.i. és una cua no buida
   (<=> primer_node != NULL) */
/* Post: El resultat és el primer element afegit al p.i. */
{
    return primer_node->info;
}
bool es_buida() const
/* Pre: cert */
/* Post: El resultat indica si el p.i. és una cua
   buida o no */
{
    return longitud==0;
}
```

Mètodes públics: consultors (II)

```
int mida() const
/* Pre: cert */
/* Post: El resultat és el nombre d'elements
        del p.i. */
{
    return longitud;
}
```

Mètodes públics: modificadors (I)

```
void demanar_torn (const T& x)
/* Pre: cert */
/* Post: El p.i. és com el p.i. original amb x
         afegit com a darrer element */
{
    node_cua* aux;
    aux = new node_cua; // reserva espai nou element
    aux->info = x;
    aux->seguent = NULL;
    if (primer_node == NULL) primer_node = aux;
    else ultim_node->seguent = aux;
    ultim_node = aux;
    ++longitud;
}
```

Mètodes públics: modificadors (II)

```
void avancar ()
/* Pre: El p.i. és una cua no buida
   (<=> primer_node != NULL) */
/* Post: El p.i. és com el p.i. original però sense
   el primer element afegit al p.i. original */
{
    node_pila* aux;
    aux = primer_node; // guardar abans d'avançar
    if (primer_node->seguent == NULL) {
        primer_node = NULL;
        ultim_node = NULL;
    }
    else primer_node = primer_node->seguent; // avança
    delete aux; // allibera l'espai de l'antic front
    —longitud;
}
```

Mètodes públics: modificadors (III)

```
void c_buida()  
/* Pre: cert */  
/* Post: El p.i. passa a ser una cua sense elements  
        i qualsevol contingut anterior del p.i. ha estat  
        esborrat */  
{  
    esborra_node_cua(primer_node);  
    longitud = 0;  
    primer_node = NULL;  
    ultim_node = NULL;  
}
```

Mètodes públics: modificadors (IV)

Redefinició de l'operador assignació

```
Cua& operator=(const Cua& original)
/* Pre: cert */
/* Post: El p.i. passa a ser una còpia d'original
   i el contingut anterior del p.i. ha estat
   esborrat (excepte si el p.i. i original ja eren
   el mateix objecte) */
{
    if (this != &original) { // si no són el mateix objecte
        longitud = original.longitud;
        esborra_node_cua(primer_node); // si no, leak!
        primer_node = copia_node_cua(original.primer_node,
                                      ultim_node);
    }
    return *this;
}
```

Mètodes privats: per copiar cadenes de nodes

```
static node_cua* copia_node_cua(node_cua* m, node_cua* &u)
/* Pre: cert */
/* Post: Si m és NULL, el resultat i u són NULL; sinó,
   el resultat apunta al primer node d'una cadena
   de nodes que són còpia de la cadena que té el
   node apuntat per m com a primer, i u apunta a
   l'últim node */
{
    node_cua* n;
    if (m==NULL) {n = NULL; u = NULL;}
    else {
        n = new node_cua;
        n->info = m->info;
        n->seguent = copia_node_cua(m->seguent, u);
        if (n->seguent == NULL) u = n;
    }
    return n;
}
```

Mètodes privats: per esborrar cadenes de nodes

```
static void esborra_node_cua(node_cua* m)
/* Pre: cert */
/* Post: No fa res si m és NULL, sinó allibera
    espai dels nodes de la cadena que té el node
    apuntat per m com a primer */
{
    if (m != NULL) {
        esborra_node_cua(m->seguent);
        delete m;
    }
}
```